

Contents

1. PSF Generation

- A. FLFM PSF simulation
- B. FLFM PSF process
 - a. FLFM PSF shift
 - b. FAST PSF generation

2. SADA Library Generate

- A. Apply Spatial-Angular Degradation Model

3. FAST Fusion Training

- A. Environment installation
- B. FAST Fusion Network Training
 - a. Training data generation
 - b. Training && Validation

4. FAST Deconvolution

1. PSF generation

A. FLMF PSF simulation

Note: Before PSF simulation, users need to download Matlab first.

We use wave-optics-based simulation code to generate designed FLMF PSF. This PSF simulation code is modified from JiaShu, the main function named 'main_FLMF_PSFgenerate.m' is located in the subfolder './FAST_PSF_generate' as shown in Fig1.

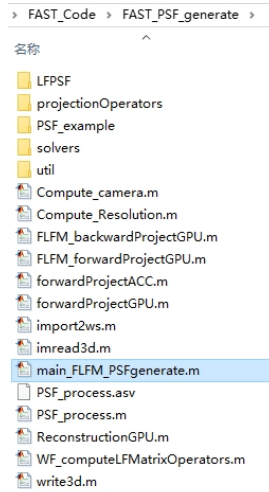


Fig 1

Users need to set their customized parameters in 'main_FLMF_PSFgenerate.m'. For example, we use the following optics design:

Objective: 60x/1.5	---- > config.NA=1.5; config.M=60;
Tube lens: Thorlabs TTL 180	---- > config.f1=180000;
Fourier lens: ACT508-250	---- > config.f2=250000;
MLA(f=120mm,pitch=3250um)	---- > config.lenPitch=3250; Config.fm=120000;
sCMOS(pixel size=6.5um)	---- > config.spacingPixel=3250/6.5=499;

After calculation, the PSF matrixes are automatically saved in the corresponding folder, e.g. 'PSF_example' named 'PSF_[60 1.5 z-5-5 z0.1um].tif'.

B. FLMF PSF process

The original FLMF PSF needs to be processed before it can be used for subsequent FAST Fusion and FAST Deconvolution. This is achieved by **running 'PSF_process.m' located at './FAST_PSF_generate'**, which will automatically calculate and save four types of PSF: raw FLMF PSF, shifted FLMF PSF for SADA Library generation, upsampled FLMF PSF and FAST PSF for FAST Deconvolution.

Before running 'PSF_process.m', users need to set the parameters according to

actual values:

- `crop_size`: the cropping size of FLFM PSF
- `view_number`: the number of views of FLFM system
- `z_number`: the number of layers of the PSF
- `z_start/z_finish`: the first layer and the last layer of the PSF
- `centerPSF_size_xy`: the cropping size of shifted FLFM PSF
- `scale`: the upsampling factor of PSF
- `z_start_fit_coordi/z_finish_fit_coordi/edge_trim`: determine the fitting range for generating FAST PSF.

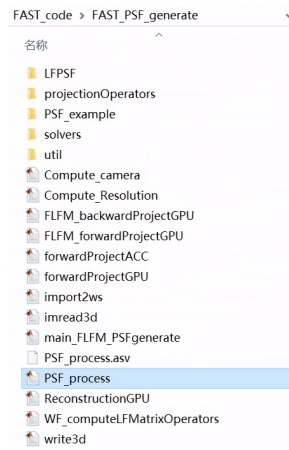


Fig 2

a. FLFM PSF shift

Move the kernel of each layer of the FLFM PSF from each view to the center of the

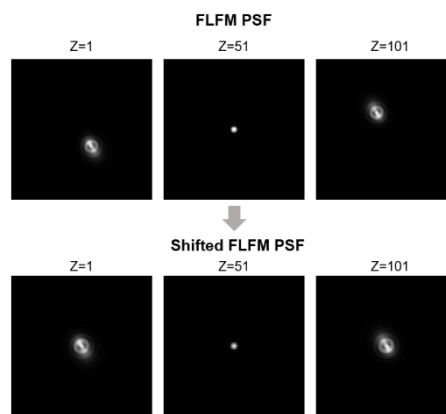


Fig 3

image, obtaining the shifted FLFM PSF, which is used for subsequent generation of the SADA library.

b. FAST PSF generation

Extract the peak position of each layer of the upsampled FLFM PSF, generate a peak position mask through linear fitting, and convolve it with the FA-PSF layer by layer to generate the FAST PSF with FA resolution, as shown in Figure 4.

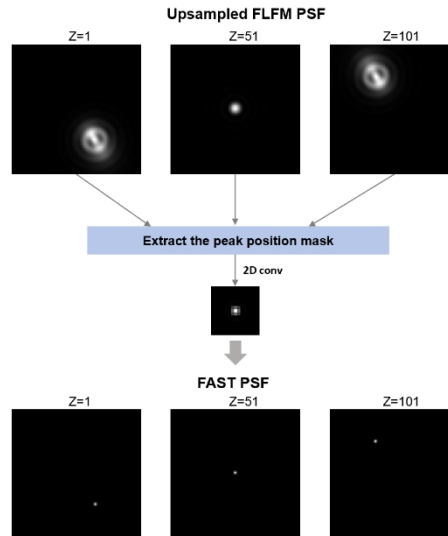


Fig 4

Run 'PSF_process.m', four types of PSF will be saved separately in the corresponding



folders, along with the corresponding .mat files.

Fig 5

2. SADA Library generate

A. Apply Spatial-Angular Degradation Model

The code of Spatial-Angular Degradation is located in the subfolder './FAST_Fusion/examples/ Fast_Fusion', named as '0_sada_generate.py' shown in Fig 6.

FAST_code > FAST_Fusion > examples > FAST_Fusion	
名称	修改日期
.idea	2026/7/1 11:00
__pycache__	2026/2/17 11:04
example_data	2026/7/1 20:37
example_Trainingimgs	2026/7/2 17:48
models	2026/7/2 17:55
PSF_example	2026/7/2 17:52
training data file	2026/7/2 17:59
utils.py	2026/2/17 10:54
0_sada_generate.py	2026/7/2 17:58
1_trainingdata_generate.py	2026/7/2 17:55
2_training.py	2026/7/2 18:14
3_prediction_viewstack.py	2026/7/2 17:55

Fig 6

Put ground truth image in './example_Trainingimgs/FA', enter actual paths of center-shifted PSF as Fig 8.

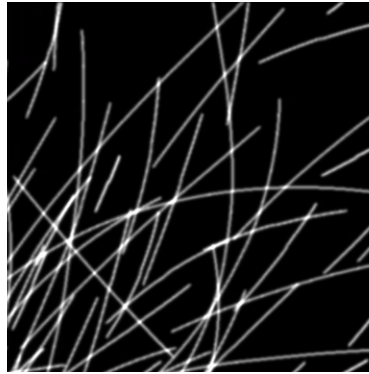


Fig 7

```
path = r'..\PSF_example\crop_shift'
root = r'..\example_Trainingimgs'
img_path = os.path.join(root, 'FA')
img_path_de = os.path.join(root, 'FA')
save_path_HR = os.path.join(root, 'FA_train')
save_path_LR = os.path.join(root, 'SADA')
mkdir(save_path_HR)
mkdir(save_path_LR)
```

Fig 8

Run "0_sada_generate.py", the Spatial-Angular Degradation Library and corresponding paired ground truth images will be saved in './example_Trainingimgs/SADA' and './example_Trainingimgs/FA_train' separately.

3. FAST Fusion Training

A. Environment installation

The source code of FAST Fusion Training is located at './FAST_Fusion/ examples/

Fast_Fusion'.

FAST Fusion Training required running environment installation.

Critical: We have tested the codes at the following settings

- Operation system: Windows 10
- GPU: RTX 2080 Ti & RTX 3090 & RTX 4090
- Cuda v11.1 and cudnn v8.2.0.
- Tensorflow 1.15

Our FAST Fusion Training code is built on Tensorflow 1.x, which is modified from source code of the article Content-aware image restoration: pushing the limits of fluorescence microscopy.

The official version of TF would not compatible with Ampere GPU (like RTX 30 series or latter), because Google no longer provide releases on TF 1.x branch after the release of TF 1.15. Here, we used the third-party [Tensorflow wheel](#) in Windows. This file and corresponding Cuda & cudnn were packed and uploaded in [Google Drive](#). For Linux, users can install the dependencies as listed in “create_environment.yml” but please delete the Tensorflow wheel in the “.yml” file. Users need to install [TF 1.x of Nvidia version](#).

i. Anaconda installation.

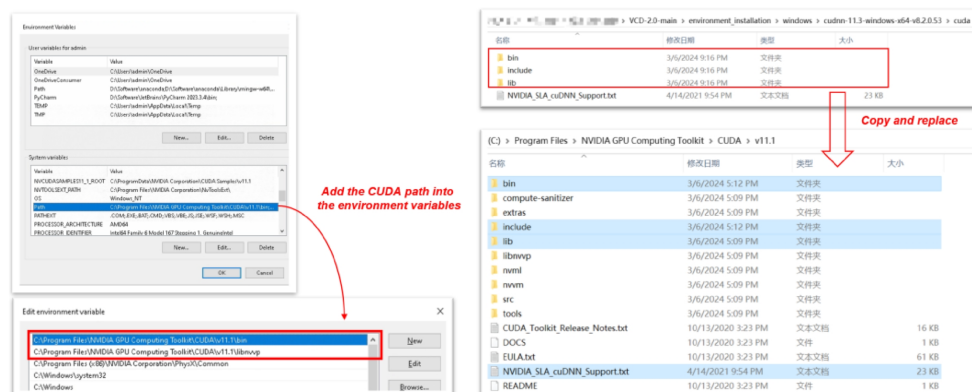
We recommend to use Anaconda to manage different running environments.

Users can download this software at <https://docs.conda.io/en/latest/>.

ii. Install Cuda and cudnn.

For convenience, we uploaded Cuda & cudnn file at [Google Drive](#). After downloading finished, unzip these files into the sub-folder ‘./environment_installation/ windows’

- ① Click ‘cuda_11.1.1_456.81_win10.exe’ and install.
- ② Add the CUDA path into “environment variables”
- ③ Unzip ‘cudnn-11.3-windows-x64-v8.2.0.53.zip’ and copy three sub-folders (‘bin’, ‘include’, ‘lib’) into the CUDA installation folder (e.g., ‘C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.1’)



iii. Dependencies installation.

We provide a package list for required dependencies installation, which is located at `./FAST_Fusion/create_environment.yml`. To use this, open the conda console in the installation file folder and run the following command inside the console:

```
>> conda env create -f create_environment.yml
```

Waiting for the automatic installation (about several minutes). When installation was finished, the prompt will print "done".

Note: Make sure "create_environment.yml" and "tensorflow-1.15.4+nv-cp38-cp38-win_amd64.whl" are in the same folder. If there exists no ".whl" file, please download from [Google Drive](#).

B. FAST Fusion Network Training

a. Training data generation

Check the raw data paths in `'1_trainingdata_generate.py'`. The `'source_dir'` and `'target_dir'` correspond to the low-resolution SADA images and the high-resolution FA images respectively. Set a reasonable patch size, the paired images will be cropped into several image blocks according to the `patch_size`. The default size is 64×64 .

Run `'1_trainingdata_generate.py'`, the training dataset composed of small image blocks will be saved as a `.npz` file in `'./training data file'`.

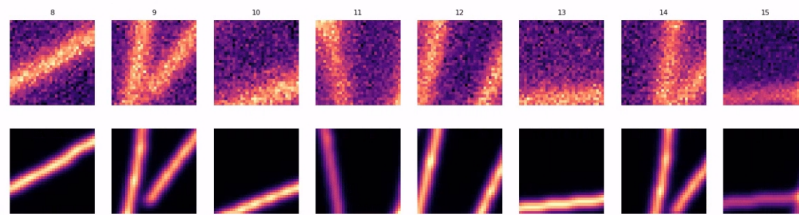


Fig 9

b. Training && Validation

Enter code `'2_training.py'`, load the training data in `.npz` format generated in the previous step. Fast Fusion is trained using the modified CARE network. Users can set the training parameters in the Config file.

Run `'2_training.py'` to start training, the trained model will be saved in the `'./models'` subfolder.

After the training is completed, '3_prediction_viewstack.py' is used to perform the inference of the network. This program supports inferencing that is input in the view-stack format. Select the appropriate up-sampling rate, model and ST data path.

```
scale_index,ckpt_name] = [1,'model_example.npz']
odel = CARE(config=None, name=ckpt_name, basedir='models')
ile_folder_name=r'..\example_data\ST'
ave_folder=os.path.join(file_folder_name,ckpt_name)
kdir(save_folder)
```

Fig 10

Run '3_prediction_viewstack.py', the results of fast fusion will be saved in a subfolder under the original ST data path.

4. FAST Deconvolution

The main function of FAST Deconvolution is located in the subfolder './FAST_Deconvolution' named 'batch_deconve.m'. The procedure for deconvolution is accomplished through the 'deconvRL_DAO.m' function.

Select the 'H.mat' file from 'FAST_PSF_generate\PSF_example\crop_upsample' and the 'Ht.mat' file from 'FAST_PSF_generate\PSF_example\crop_FAST' respectively as the PSF for the forward projection and the inverse projection of deconvolution, in order to match the sampling rate and resolution of the image.

```
H=load('..\FAST_PSF_generate\PSF_example\crop_upsample\crop\H_scale[60 1.5 -5-5 0.1].mat');
H=H.H_scale;
Ht=load('..\FAST_PSF_generate\PSF_example\crop_FAST\Ht_fast[60 1.5 -5-5 0.1].mat');
Ht=Ht.Ht_FAST;
```

Fig 11

Run 'batch_deconve.m', select the fast fusion images that need to be reconstructed in the popped-up window. The reconstructed images will be saved in a subfolder under the input data path.